

第9章 聚类

浙江理工大学 曹丽

办公室 10-541

无监督学习概述

□ 监督学习和无监督学习的区别

□ 监督学习

在一个典型的监督学习中，训练集有标签 y ，我们的目标是找到能够区分正样本和负样本的决策边界，需要据此拟合一个假设函数。

□ 无监督学习

与此不同的是，在无监督学习中，我们的数据没有附带任何标签 y ，无监督学习主要分为聚类、降维、关联规则和推荐系统等方面。

无监督学习概述

聚类和分类的区别

- 聚类与分类最主要的差别是聚类的样本不具有类别标号，而分类的样本具有类别标号。
- 聚类是**无监督学习(unsupervised learning)**，而分类是**有监督学习(supervised learning)**。因此，分类里有训练和测试，而聚类没有训练。
- 尽管分类是识别对象组类别的有效手段，但需要高昂的代价收集和标记训练样本集。因此，聚类提供了一种新的处理模式：先把数据集划分为组，然后给有限的组指定类别标号。

无监督学习概述

主要的无监督学习方法

- ✓ 聚类
 - ✓ 如何将教室的学生按照爱好和身高划分为5类；
- ✓ 降维
 - ✓ 如何将高维空间的数据映射到低维空间
- ✓ 关联规则
 - ✓ 很多买尿布的男顾客，同时买了啤酒，从中可以找出什么规律来提高超市的销售额？
- ✓ 推荐系统
 - ✓ 很多顾客经常上网购物，根据他们浏览商品的习惯，给他们推荐什么商品呢？

无监督学习概述

聚类

☐ 主要算法

- ✓ k-means、密度聚类、层次聚类

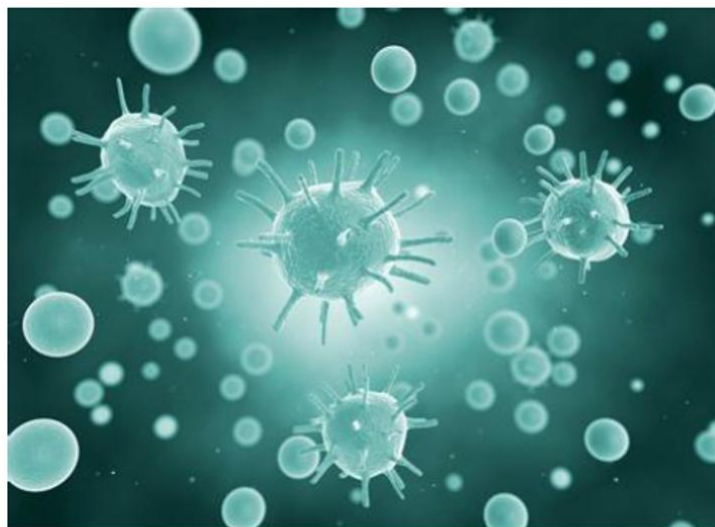
☐ 主要应用

- ✓ 市场细分、文档聚类、图像分割、聚类分析、特征学习或者词典学习、确定犯罪易发地区、保险欺诈检测、公共数据交通分析、IT资产集群、客户细分、识别癌症数据、搜索引擎应用、医疗应用、药物活性预测。

无监督学习概述—聚类的案例

1. 医疗

医生可以使用聚类算法来发现疾病，以甲状腺疾病为例。我们对包含甲状腺疾病和非甲状腺疾病的数据集应用无监督学习时，可以使用聚类算法来识别甲状腺疾病数据集。



无监督学习概述—聚类的案例

2. 市场细分

为了吸引更多的客户，每家公司都在开发易于使用的功能和技术。为了了解客户，公司可以使用聚类。聚类将帮助公司了解用户群，然后对每个客户进行归类。这样，公司就可以了解客户，发现客户之间的相似之处，并对他们进行分组。



无监督学习概述—聚类的案例

2. 市场细分

为了吸引更多的客户，每家公司都在开发易于使用的功能和技术。为了了解客户，公司可以使用聚类。聚类将帮助公司了解用户群，然后对每个客户进行归类。这样，公司就可以了解客户，发现客户之间的相似之处，并对他们进行分组。



无监督学习概述—聚类的案例

3. 搜索引擎

百度是人们使用的搜索引擎之一。举个例子，当我们搜索一些信息，如在某地的超市，百度将为我们提供不同的超市的选择。这是聚类的结果，提供给你的结果就是聚类的相似结果。



无监督学习概述—聚类的案例

4. 社交网络

比如在社交网络的分析上。已知你朋友的信息，比如经常发email的联系人，或者是你的微博好友、微信的朋友圈，我们可运用聚类方法自动地给朋友进行分组，做到让每组里的人们彼此都熟识。

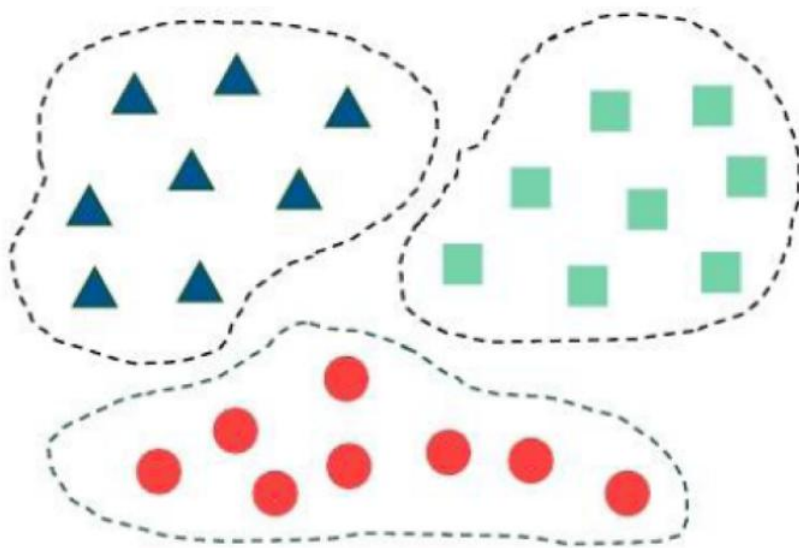


无监督学习概述—聚类的案例

在实际应用场景中，**聚类分析**常用来对目标用户群体分类，把目标群体分为几个特征鲜明的细分群体，从而可以在运营活动中对这些不同群体开展差异化服务。另外聚类分析也常用来发现异常和孤立点，比如，在电子商务网站的购物交易中，通过聚类发现异常的有欺骗风险的交易行为。

聚类算法

思想： 将数据样本划分为若干个通常不相交的“簇”（cluster），既可以作为一个单独的过程（用于寻找数据内在的分布结构），也可作为分类其他学习任务的前驱过程。



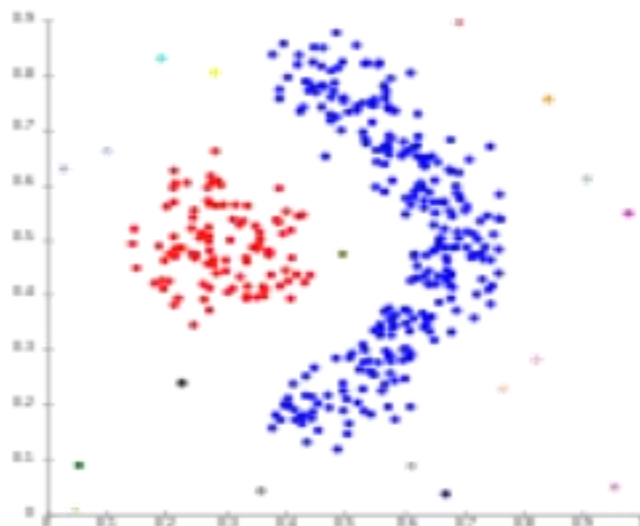
聚类算法

目标： 指将样本分到不同的组中使得同一组中的样本差异尽可能的小，而不同组中的样本差异尽可能的大。

聚类得到的不同的组称为簇(cluster)。

一个好的聚类方法将产生以下的聚类

- 最大化类中的相似性
- 最小化类间的相似性



聚类算法—性能度量

聚类性能度量：亦称聚类“有效性指标”

■ 外部指标

- 将聚类结果与某个“参考模型”进行比较
- 如Jaccard系数，FM指数，Rand指数

■ 内部指标

- 直接考察聚类结果而不同任何参考模型
- 如DB指数，Dunn指数等

基本想法：

- “簇内相似度” 高
- “簇间相似度” 低

聚类算法—聚类效果



聚类的“好坏”不存在绝对标准

**the goodness of clustering depends on
the opinion of the user**

聚类算法—聚类效果

聚类的故事：

老师拿来苹果和梨，让小朋友们分成两份。

小明把大苹果大梨放一起，小个头的放一起，老师点头，
嗯，**体量感**

小芳把红苹果挑出来，剩下的放一起，老师点头，**颜色感**。

小武的结果？不明白。小武掏出眼镜：最新款，能看到水果里有几个籽，左边这堆单数，右边双数。

老师们很高兴：新的聚类算法诞生了。

聚类也许是机器学习中的“新算法”出现最多，最快的领域，
总能找到一个新标准，使以往算法对它无能为力。

常见的聚类方法

原型聚类

- 亦称“基于原型的聚类”
- 假设：聚类结构能通过一组原型刻画
- 过程：先对原型初始化，然后对原型进行迭代更新求解
- 代表：**K均值聚类**，高斯混合聚类

密度聚类

- 亦称“基于密度的聚类”
- 假设：聚类结构能通过样本分布的紧密程度确定
- 过程：从样本密度的角度来考察样本之间的可连接性，并基于可连接样本不断扩展聚类簇
- 代表：**DBSCAN**，OPTICS，DENCLUE

层次聚类

- 假设：能够产生不同粒度的聚类结果
- 过程：在不同层次对数据集进行划分，从而形成属性的聚类结构
- 代表：**AGNES（自底向上）**，DIANA（自顶向下）

K-mean均值聚类

K-means算法是一种无监督学习方法，是最普及的聚类算法，算法使用一个没有标签的数据集，然后将数据聚成不同的组。

K-means算法具有一个迭代过程，在这个过程中，数据集被分组成若干个预定义的不重叠的聚类或者子组，使簇的内部点尽可能相似，同时试图保持簇在不同的空间，它将数据点分配给簇，以便簇的质心和数据点之间的平方距离之和最小，在这个位置，簇的质心是簇中数据点的算法平均值。

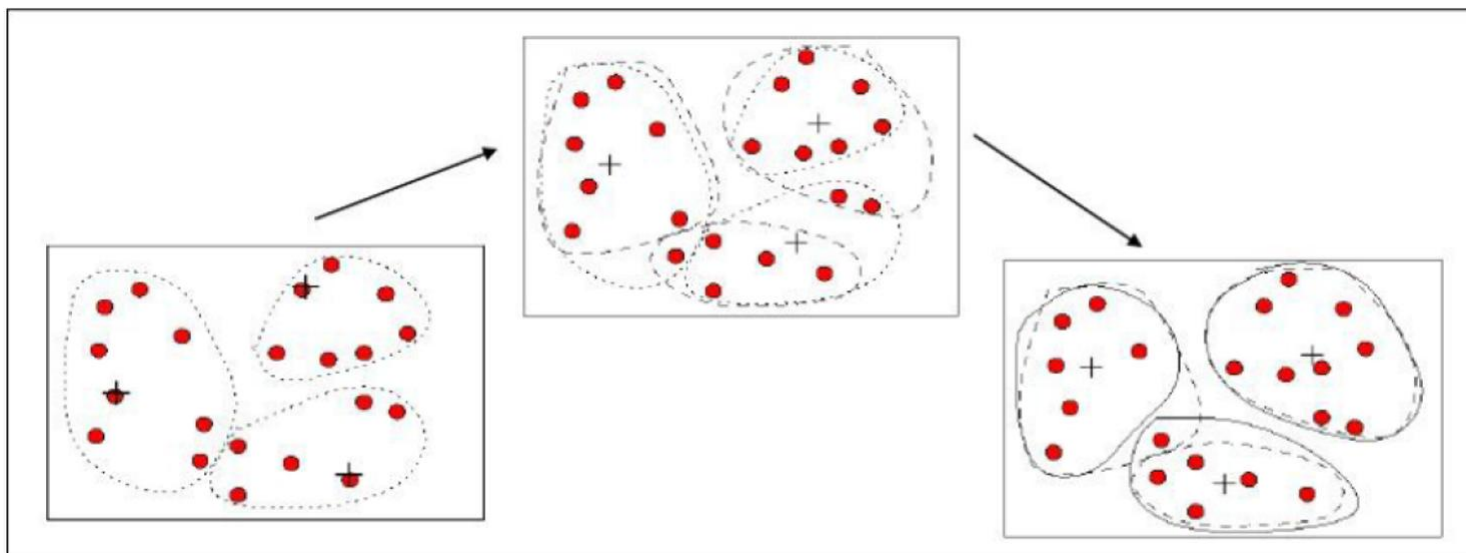
K-mean均值聚类

每个簇以该簇中所有样本点的“均值”表示

Step1: 随机选取k个样本点作为簇中心

Step2: 将其他样本点根据其与其与簇中心的距离，划分给最近的簇

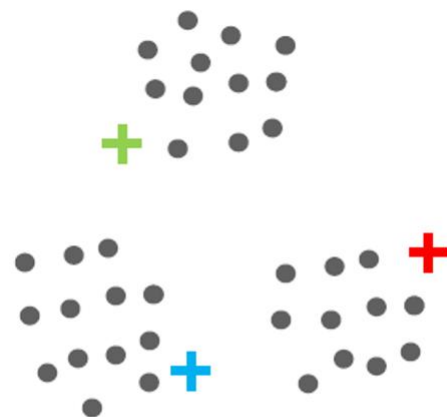
Step3: 若所有簇中心未发生改变，则停止；否则执行Step2



K-mean均值聚类算法流程

K-means算法流程

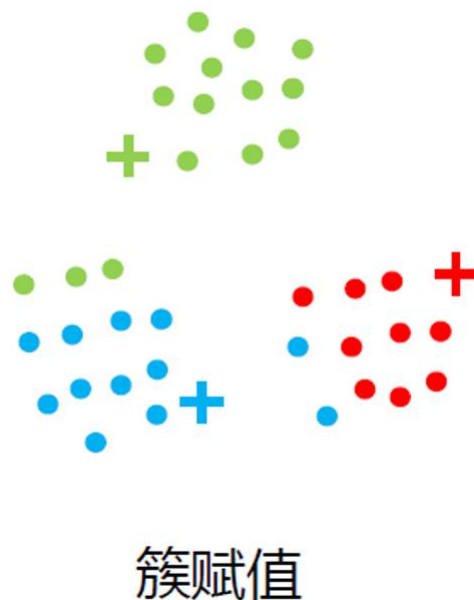
首先，初始化称为簇质心的任意点。初始化时，必须注意簇的质心必须小于训练数据点的数据。因为该算法是一种迭代算法，接下来的两个步骤是迭代执行的。



初始化质心

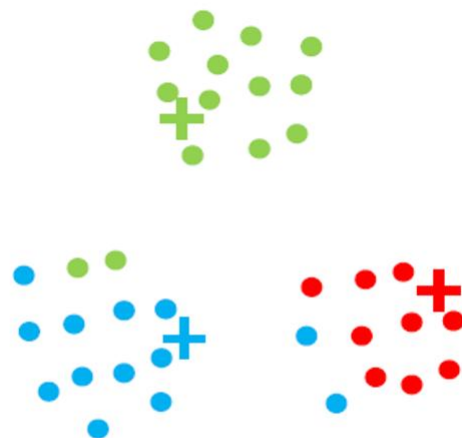
K-mean均值聚类算法流程

初始化后，遍历所有数据点，
计算所有质心与数据点的距离。
现在，这些簇将根据与质心的
最小距离而形成。在本例中，
数据分为3个簇（ $k=3$ ）



K-mean均值聚类算法流程

第三步：移动质心，因为上面步骤中形成的簇没有优化，所以需要形成优化簇。为此，我们需要迭代地将质心移动到一个新的位置。取一个簇的数据点，计算它们的平均值，然后将该簇的质心移动到这个新位置。对所有其他的簇重复相同的步骤。



迭代更新

K-mean均值聚类算法策略

- 定义样本与其所属类的中心之间的距离的总和为损失函数，即

$$W(C) = \sum_{l=1}^k \sum_{C(i)=l} \|x_i - \bar{x}_l\|^2$$

- $\bar{x}_l = (\bar{x}_{1l}, \bar{x}_{2l}, \dots, \bar{x}_{ml})^T$ 第*l*个类的均值或中心，
- 函数W（C）也称为能量，表示相同类中的样本相似的程度。

上述两个步骤是迭代进行的，直到质心停止移动，即它们不再改变自己的位置，并且成为静态的。一旦这样做，K-均值算法被称为收敛。

K-mean均值聚类优化

K均值聚类就是求解最优化问题：

$$\begin{aligned} C^* &= \arg \min_C W(C) \\ &= \arg \min_C \sum_{l=1}^k \sum_{C(i)=l} \|x_i - \bar{x}_l\|^2 \end{aligned}$$

相似的样本被聚到同类时，损失函数值最小，这个目标函数的最优化能达到聚类的目的。

K-mean均值算法

首先，对于给定的中心值

$$\min_C \sum_{l=1}^k \sum_{C(i)=l} \|x_i - m_l\|^2$$

相似的样本被聚到同类时，损失函数值最小，这个目标函数的最优化能达到聚类的目的。

K-mean均值算法

输入: n 个样本的集合 X ;

输出: 样本集合的聚类 C^* 。

(1) 初始化。令 $t = 0$, 随机选择 k 个样本点作为初始聚类中心 $m^{(0)} = (m_1^{(0)}, \dots, m_l^{(0)}, \dots, m_k^{(0)})$ 。

(2) 对样本进行聚类。对固定的类中心 $m^{(t)} = (m_1^{(t)}, \dots, m_l^{(t)}, \dots, m_k^{(t)})$, 其中 $m_l^{(t)}$ 为类 G_l 的中心, 计算每个样本到类中心的距离, 将每个样本指派到与其最近的中心的类中, 构成聚类结果 $C^{(t)}$ 。

(3) 计算新的类中心。对聚类结果 $C^{(t)}$, 计算当前各个类中的样本的均值, 作为新的类中心 $m^{(t+1)} = (m_1^{(t+1)}, \dots, m_l^{(t+1)}, \dots, m_k^{(t+1)})$ 。

(4) 如果迭代收敛或符合停止条件, 输出 $C^* = C^{(t)}$ 。

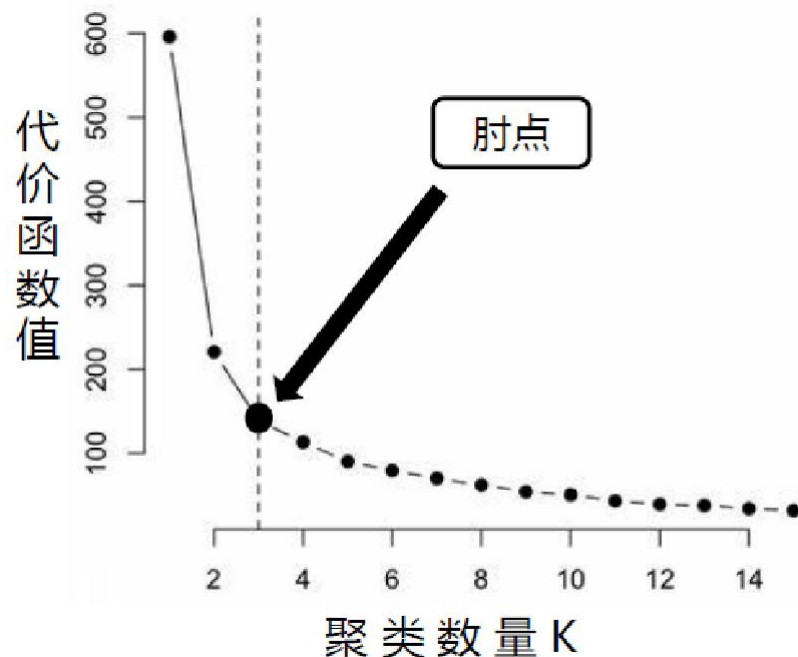
否则, 令 $t = t + 1$, 返回步 (2)。 ■

k 均值聚类算法的复杂度是 $O(mnk)$, 其中 m 是样本维数, n 是样本个数, k 是类别个数。

K-mean均值算法

K值的选择

现在我们需要找到簇的数量。通常通过**肘部法则**进行计算。我们可能会得到一条类似人肘部的曲线。右图中，代价函数的值会迅速下降，在 $k=3$ 时，达到一个肘点。在此之后，代价函数的值就会下降地非常慢，所以，我们选择 $K=3$ 。这个方法叫做“肘部法则”。



K-mean均值算法—例子

给定含有5个样本的集合

$$X = \begin{bmatrix} 0 & 0 & 1 & 5 & 5 \\ 2 & 0 & 0 & 0 & 2 \end{bmatrix}$$

试用k均值聚类算法将样本聚到2个类中。

K-mean均值算法—例子

(1) 选择两个样本点作为类的中心。假设选择 $m_1^{(0)} = x_1 = (0, 2)^T$, $m_2^{(0)} = x_2 = (0, 0)^T$ 。

(2) 以 $m_1^{(0)}$, $m_2^{(0)}$ 为类 $G_1^{(0)}$, $G_2^{(0)}$ 的中心, 计算 $x_3 = (1, 0)^T$, $x_4 = (5, 0)^T$, $x_5 = (5, 2)^T$ 与 $m_1^{(0)} = (0, 2)^T$, $m_2^{(0)} = (0, 0)^T$ 的欧氏距离平方。

对 $x_3 = (1, 0)^T$, $d(x_3, m_1^{(0)}) = 5$, $d(x_3, m_2^{(0)}) = 1$, 将 x_3 分到类 $G_2^{(0)}$ 。

对 $x_4 = (5, 0)^T$, $d(x_4, m_1^{(0)}) = 29$, $d(x_4, m_2^{(0)}) = 25$, 将 x_4 分到类 $G_2^{(0)}$ 。

对 $x_5 = (5, 2)^T$, $d(x_5, m_1^{(0)}) = 25$, $d(x_5, m_2^{(0)}) = 29$, 将 x_5 分到类 $G_1^{(0)}$ 。

K-mean均值算法—例子

(3) 得到新的类 $G_1^{(1)} = \{x_1, x_5\}$, $G_2^{(1)} = \{x_2, x_3, x_4\}$, 计算类的中心 $m_1^{(1)}$, $m_2^{(1)}$:

$$m_1^{(1)} = (2.5, 2.0)^T, \quad m_2^{(1)} = (2, 0)^T$$

(4) 重复步骤 (2) 和步骤 (3)。

将 x_1 分到类 $G_1^{(1)}$, 将 x_2 分到类 $G_2^{(1)}$, x_3 分到类 $G_2^{(1)}$, x_4 分到类 $G_2^{(1)}$, x_5 分到类 $G_1^{(1)}$ 。

得到新的类 $G_1^{(2)} = \{x_1, x_5\}$, $G_2^{(2)} = \{x_2, x_3, x_4\}$ 。

由于得到的新的类没有改变, 聚类停止。得到聚类结果:

$$G_1^* = \{x_1, x_5\}, \quad G_2^* = \{x_2, x_3, x_4\}$$

K-mean均值算法优缺点

■ 主要优点

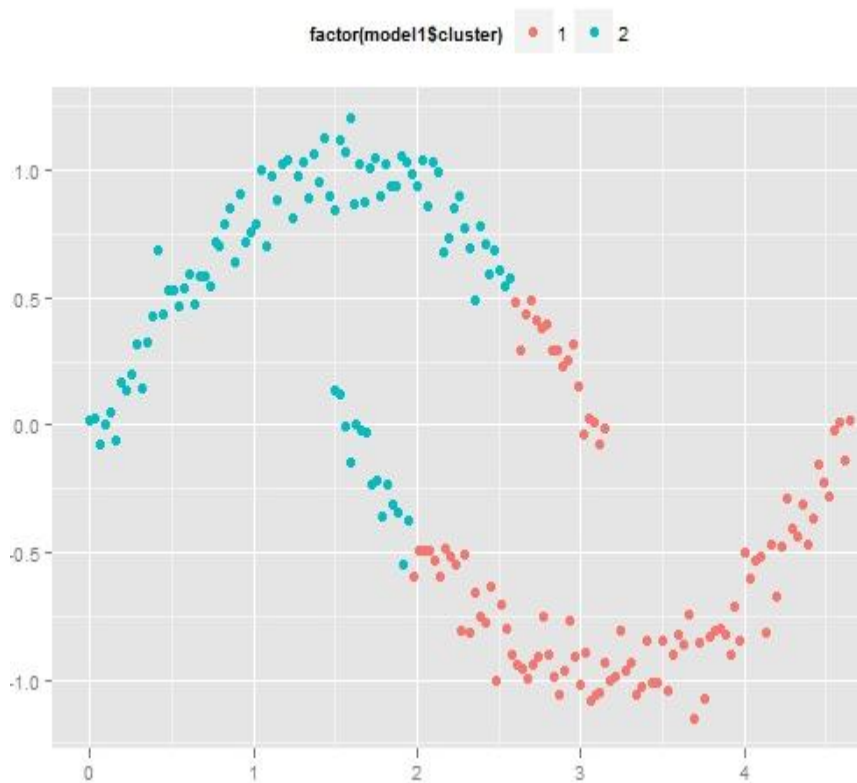
- 是解决聚类问题的一种经典算法，简单、快速。
- 对处理大数据集，该算法是相对可伸缩和高效率的。
- 当结果簇是密集的，它的效果较好。

■ 主要缺点

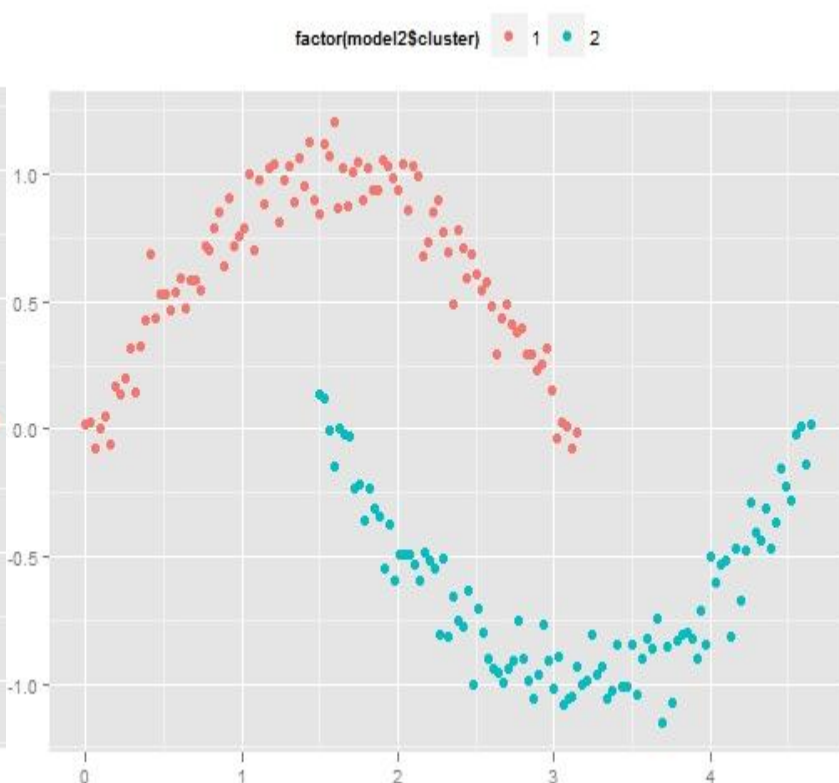
- 在簇的平均值被定义的情况下才能使用。
- 必须事先给出k（簇的数目），而且对初值敏感，对于不同的初始值，可能会导致不同结果。
- 不适合于发现非凸面形状的簇或者大小差别很大的簇。而且，它对于“噪声”和孤立点数据是敏感的。

K-mean均值算法优缺点

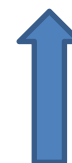
K-means (k=2)



实际型



理想型



基于密度的聚类(DBSCAN)

密度聚类

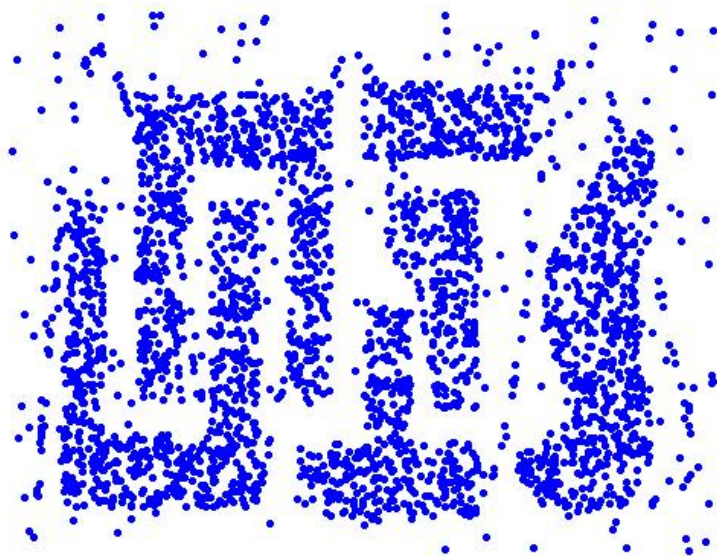
划分和层次方法旨在发现**球状簇**。他们很难发现任意形状的簇。

改进思想：将**簇**看作数据空间中由低密度区域分隔开的高密度对象区域。

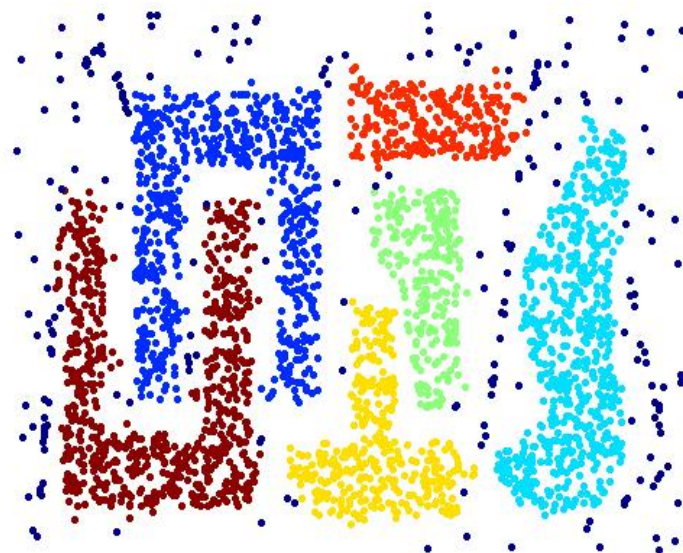
过滤噪声孤立点数据，发现任意形状的簇。

- **DBSCAN：基于高密度连通区域聚类**
- OPTICS：通过点排序识别聚类结构
- DENCLUE：基于密度分布函数的聚类

密度聚类



Original Points



Clusters

特点:

抗噪声

能处理任意形状聚类

密度聚类--DBSCAN

与划分和层次聚类方法不同，DBSCAN（Density-Based Spatial Clustering of Applications with Noise）是一个比较有代表性的基于密度的聚类算法。它将**簇定义为密度相连的点的最大集合**，能够把具有足够高密度的区域划分为簇，并可在噪声的空间数据库中发现任意形状的聚类。

密度：空间中任一点的密度是以该点为圆心，以扫描半径构成的圆区域内包含的点数目。

DBSCAN算法聚类速度快，而且能够有效处理噪声点和发现任意形状的空间聚类。和K-Means聚类相比，DBSCAN既可以适用于**凸样本集**，也可以适用于非凸样本集。

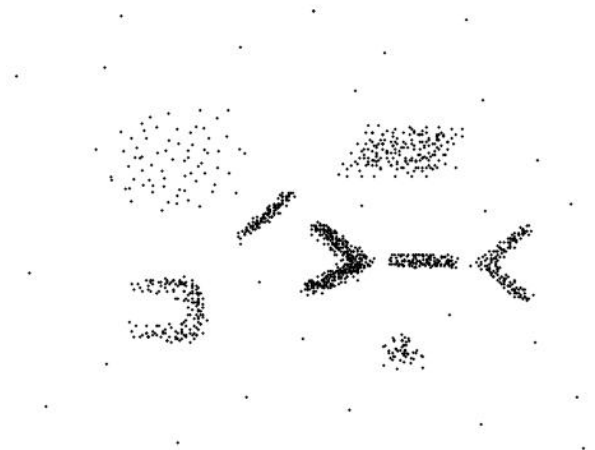
密度聚类--DBSCAN

基于密度的簇是密度相连的点的集合

主要思想

寻找被**低密度区域**分离的**高密度区域**

只要邻近区域的密度（单位大小上对象或数据点的数目）
超过某个阈值，就继续聚类



密度聚类--DBSCAN

DBSCAN 使用两个超参数：

扫描半径（eps）和最小包含点数（minPts）来获取簇的数量，而不是猜测簇的数目。

➤ 扫描半径（eps）：

用于定位点/检查任何点附近密度的距离度量，即扫描半径。

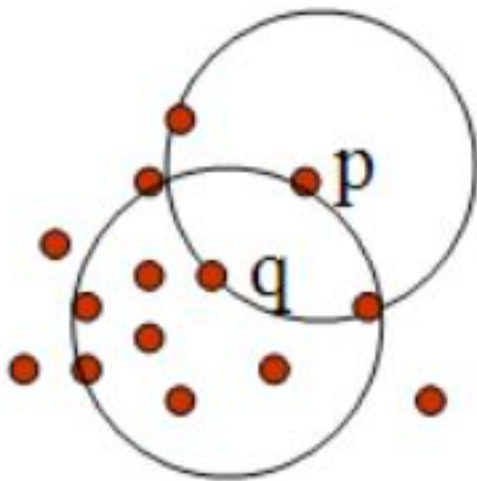
➤ 最小包含点数（minPts）：

聚集在一起的最小点数（阈值），该区域被认为是稠密的。

密度聚类--DBSCAN

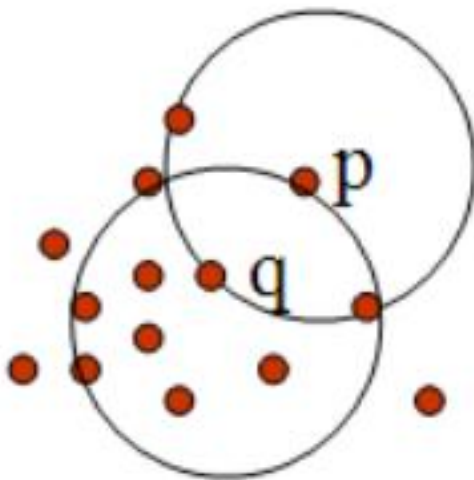
对象的 ε -邻域：给定对象在半径 ε 内的区域。

- **核心对象：**如果一个对象的 ε -邻域至少包含最小数目MinPts个对象，则称该对象为核心对象。
- 在下图中， $\varepsilon=1\text{cm}$ ，MinPts=5， q 是一个核心对象。



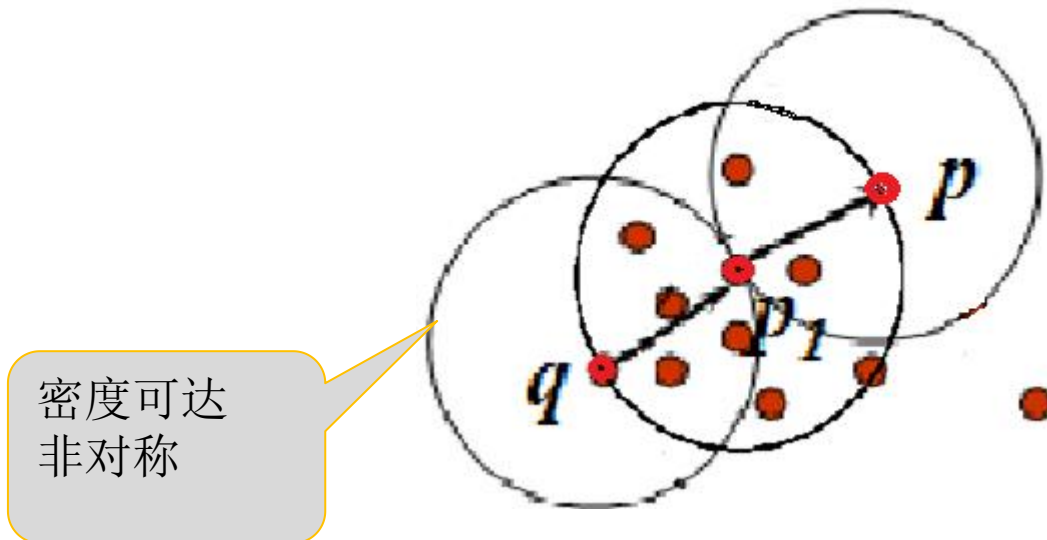
密度聚类--DBSCAN

- **直接密度**可达：给定一个对象集合D，如果p是在q的 ϵ -邻域内，而q是一个核心对象。
 - 对象p从对象q出发是直接密度可达的
- 例如，在下图中， $\epsilon=1\text{cm}$ ， $\text{MinPts}=5$ ，q是一个核心对象，对象p从对象q出发是直接密度可达的。



密度聚类--DBSCAN

- **密度可达的**：如果存在一个对象链 $p_1, p_2, \dots, p_n$, $p_1=q$, $p_n=p$, 对 $p_i \in D$, ($1 \leq i \leq n$), p_{i+1} 是从 p_i 关于 ϵ 和MinPts直接密度可达的, 则对象 p 是从对象 q 关于 ϵ 和MinPts密度可达的。
- 在下图中, $\epsilon=1\text{cm}$, $\text{MinPts}=5$, q 是一个核心对象, p_1 是从 q 关于 ϵ 和MinPts直接密度可达, p 是从 p_1 关于 ϵ 和MinPts直接密度可达, 则对象 p 从对象 q 关于 ϵ 和MinPts密度可达的。



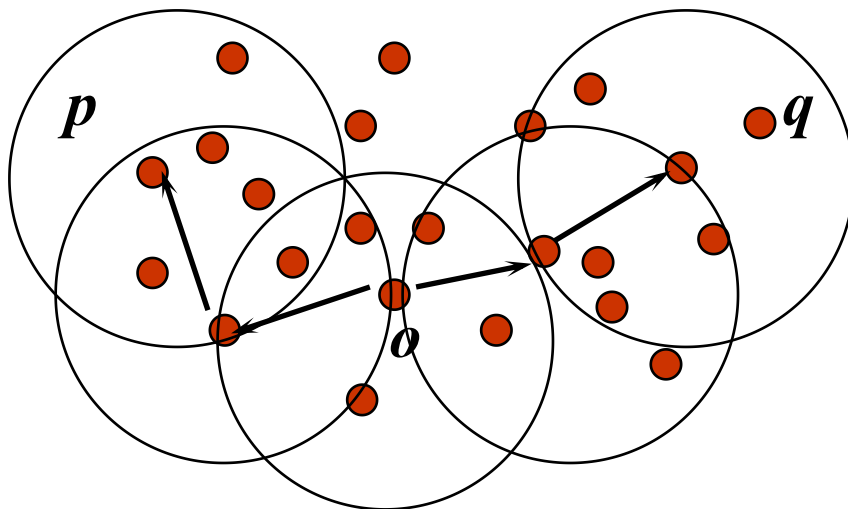
密度聚类--DBSCAN

■ 密度相连的（Density-connected）

- 如果对象集合D中存在一个对象o，使得对象p和q是从o关于 ϵ 和 MinPts密度可达的，那么对象p和q是关于 ϵ 和 MinPts 密度相连的。
- 密度相连性是一个对称的关系。

边界点：边界点不是核心点，但落在某个核心点的邻域内。

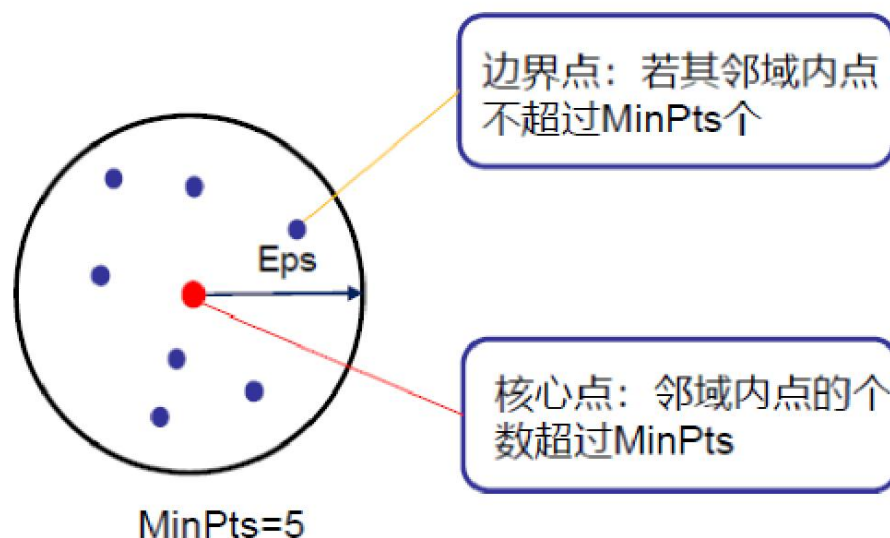
噪声：既不是边界点也不是核心点的对象



密度聚类--DBSCAN

DBSCAN 将数据分为三类：

1. **核心点**：在半径（Eps）内含有超过MinPts数目的点。
2. **边界点**：在半径（Eps）内点的数量小于MinPts数目，但是落在核心点的邻域内的点。
3. **噪声点**：既不是核心点也不是边界点。



密度聚类--DBSCAN的算法流程

输入: 样本集 $D = \{x_1, x_2, \dots, x_m\}$;

邻域参数 $(\epsilon, MinPts)$.

过程:

```
1: 初始化核心对象集合:  $\Omega = \emptyset$ 
2: for  $j = 1, 2, \dots, m$  do
3:   确定样本  $x_j$  的  $\epsilon$ -邻域  $N_\epsilon(x_j)$ ;
4:   if  $|N_\epsilon(x_j)| \geq MinPts$  then
5:     将样本  $x_j$  加入核心对象集合:  $\Omega = \Omega \cup \{x_j\}$ 
6:   end if
7: end for
8: 初始化聚类簇数:  $k = 0$ 
9: 初始化未访问样本集合:  $\Gamma = D$ 
10: while  $\Omega \neq \emptyset$  do
11:   记录当前未访问样本集合:  $\Gamma_{old} = \Gamma$ ;
12:   随机选取一个核心对象  $o \in \Omega$ , 初始化队列  $Q = \langle o \rangle$ ;
13:    $\Gamma = \Gamma \setminus \{o\}$ ;
14:   while  $Q \neq \emptyset$  do
15:     取出队列  $Q$  中的首个样本  $q$ ;
16:     if  $|N_\epsilon(q)| \geq MinPts$  then
17:       令  $\Delta = N_\epsilon(q) \cap \Gamma$ ;
18:       将  $\Delta$  中的样本加入队列  $Q$ ;
19:        $\Gamma = \Gamma \setminus \Delta$ ;
20:     end if
21:   end while
22:    $k = k + 1$ , 生成聚类簇  $C_k = \Gamma_{old} \setminus \Gamma$ ;
23:    $\Omega = \Omega \setminus C_k$ 
24: end while
```

输出: 簇划分 $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$

密度聚类使用的方法很简单，它任意选择一个没有类别的核心对象作为种子，然后找到所有这个核心对象能够密度可达的样本集合，即为一个聚类簇。接着继续选择另一个没有类别的核心对象去寻找密度可达的样本集合，这样就可以得到另一个聚类簇。已知运行到所有核心对象都有类别为止。

密度聚类--DBSCAN的算法流程

第一步，顺序扫描数据集的样本点，首先取到P1(1,2);

(1) 计算P1的邻域，计算出每一点到P1的距离，如 $d(P1, P2)=\sqrt{1+1}$
 $=1.414$;

(2) 根据每个样本点到P1的距离，计算出P1的Eps邻域为{p1, p2, p3, p13};

(3) 因为p1的Eps邻域含有4个点，大于MinPts (3)，所用p1为核心点;

(4) 以p1为核心点建立簇C1，即找出所有从p1密度可达的点;

(5) p1邻域内的点都是p1直接密度可达的点，所以都属于C1;

(6) 寻找p1密度可达的点，p2的邻域为{p1, p2, p3, p4, p13}，因为p1密度可达p2，p2密度可达p4，所以p1密度可达p4，因此，p4也属于C1;

(7) P3的邻域为{p1, p2, p3, p4, p13},p13的邻域为{p1, p2, p3, p4, p13}，p3和p13都是核心点，但是它们的邻域的点都已经在C1中。

(8) P4的邻域为{p3, p4, p13}，为核心点，其邻域内的所有点都已经被处理;

(9) 此时，以p1为核心点出发的那些密度可达的对象都全部处理完毕，得到簇C1，包含点{p1, p2, p3, p4, p13}。

密度聚类--DBSCAN的算法流程

第二步，继续顺序扫描数据集的样本点，取到p5 (5,8)。

- (1) 计算p5的邻域，计算每一点到p5的距离，如 $d(p5, p8) = \sqrt{4+1} = 2.236$;
- (2) 根据每个样本点到p5的距离，计算出p5的Eps邻域为{p5, p6, p7, p8};
- (3) 因为p5的Eps邻域含有4个点，大于MinPts (3)，所以p5为核心点;
- (4) 以p5为核心点建立簇C2，即找出所有从p5密度可达的点，可以获得簇C2，包含点{p5, p6, p7, p8};

第三步，继续顺序扫描数据集的样本点，取到p9 (9,5)

- (1) 计算p9的Eps邻域为{p9}，个数小于MinPts (3)，所用p9不是核心点;
- (2) 对p9处理结束。

第四步，继续顺序扫描数据集的样本点，取到p10 (1,12)

- (1) 计算出p10的Eps邻域为{p10, p11}，个数小于MinPts (3)，所以p10不是核心点;
- (2) 对p10处理结束。

密度聚类--DBSCAN的算法流程

第五步，继续顺序扫描数据集的样本点，取到p11(3,12).

1)计算出p11的Eps邻域为{p11, p10, p12}, 个数等于MinPts (3), 所以p11是核心点;

2) 从p12的邻域为{p11,p12}, 不是核心点;

3) 以p11为核心点建立簇C3, 包含点{p11, p10, p12}。

第六步，继续扫描数据的样本点，p12、P13都已经被处理过，算法结束。

密度聚类--DBSCAN

密度聚类算法流程

1. 将所有的点标记为核心点、标记点和噪声点；
2. 如果选择的点是核心点，则找出从该点出发的密度可达对象形成的簇
3. 如果该点是非核心点，将其指派到一个与之关联的核心点的簇中。
4. 重复以上步骤，直到所有的点都被处理过。

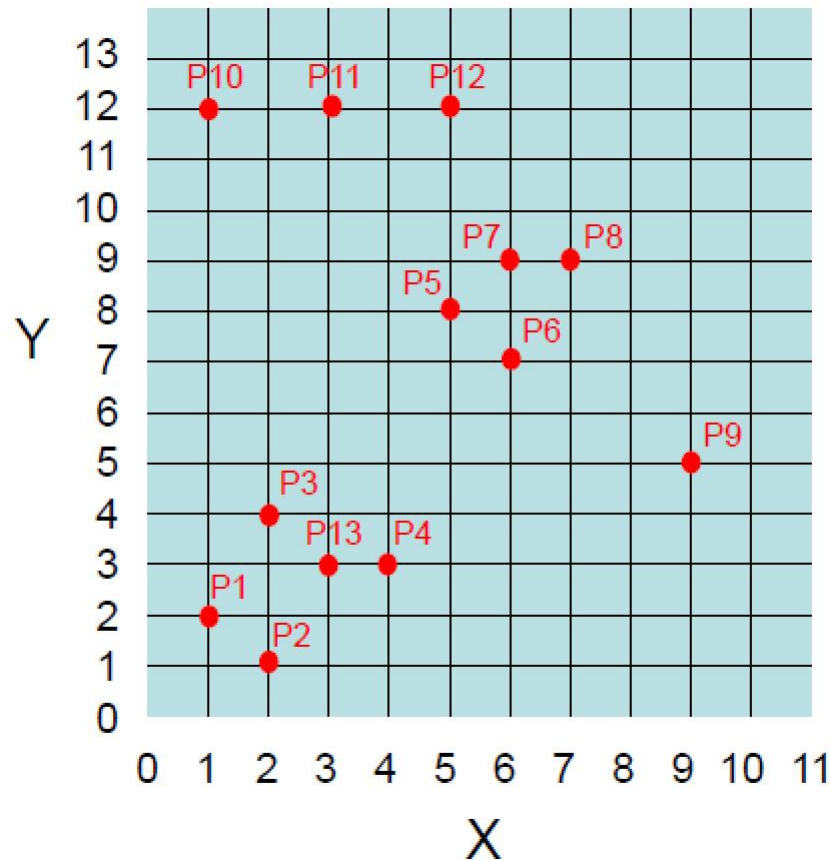
举例：有如下13个样本点，使用DBSCAN进行聚类

	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12	P13
X	1	2	2	4	5	6	6	7	9	1	3	5	3
Y	2	1	4	3	8	7	9	9	5	12	12	12	3

密度聚类--DBSCAN

密度聚类的算法流程

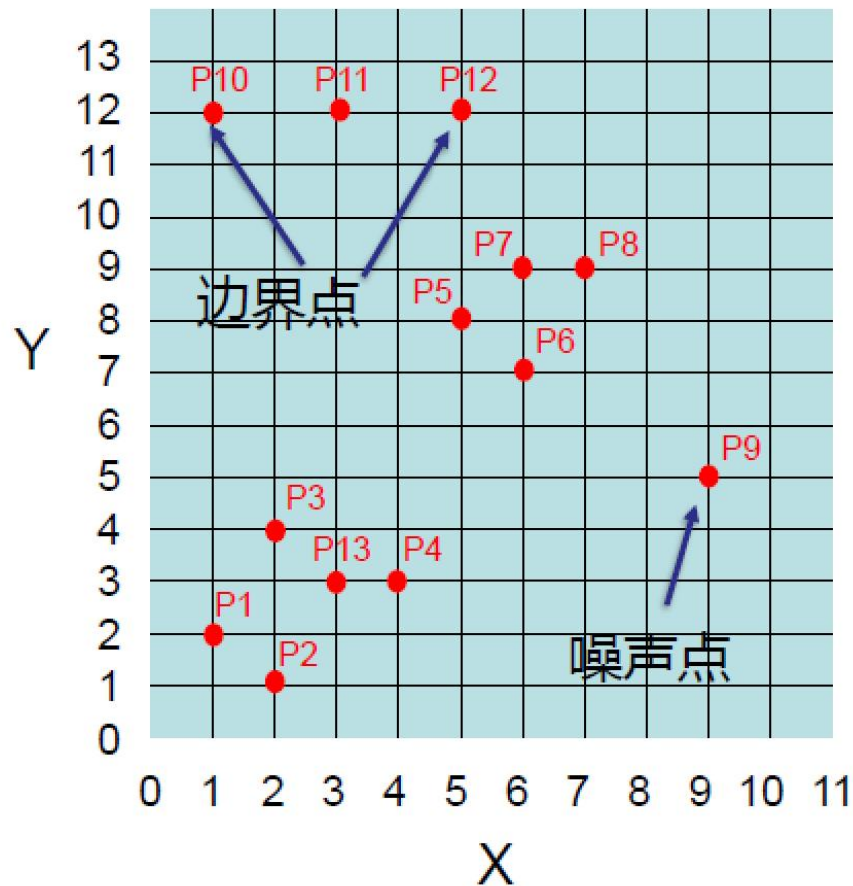
- 对每个点计算其邻域 $Eps=3$ 内的点的集合
- 集合内点的个数超过 $MinPts=3$ 的点为核心点。



密度聚类--DBSCAN

密度聚类的算法流程

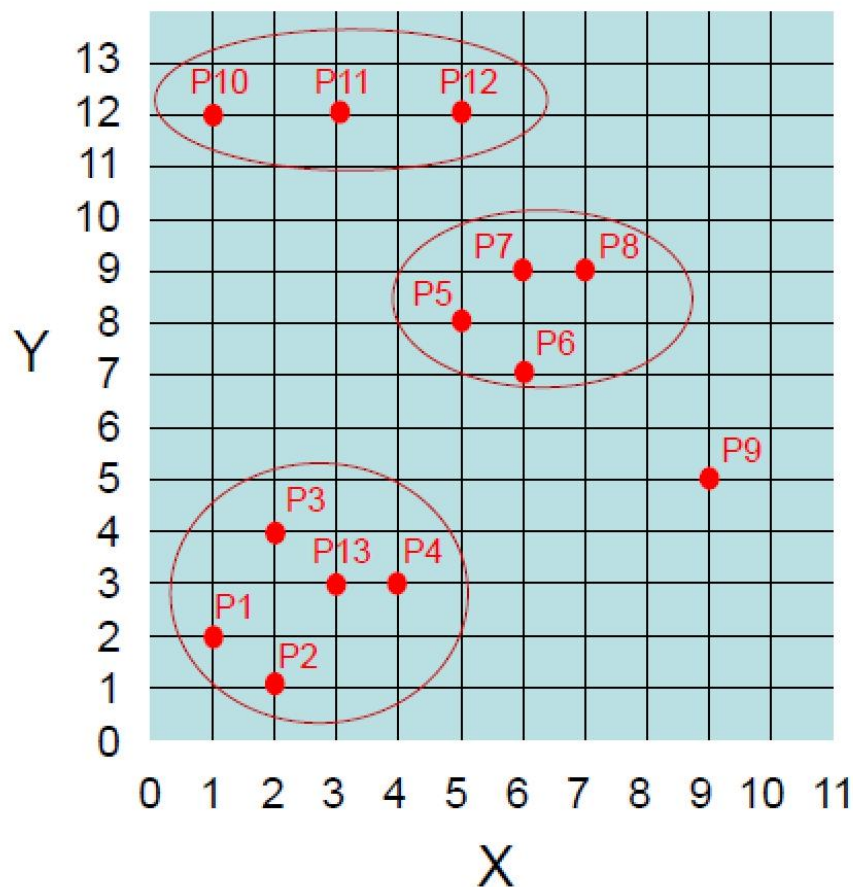
➤ 查看剩余点是否在核心点的邻域内，若在，则为边界点，否则为噪声点。



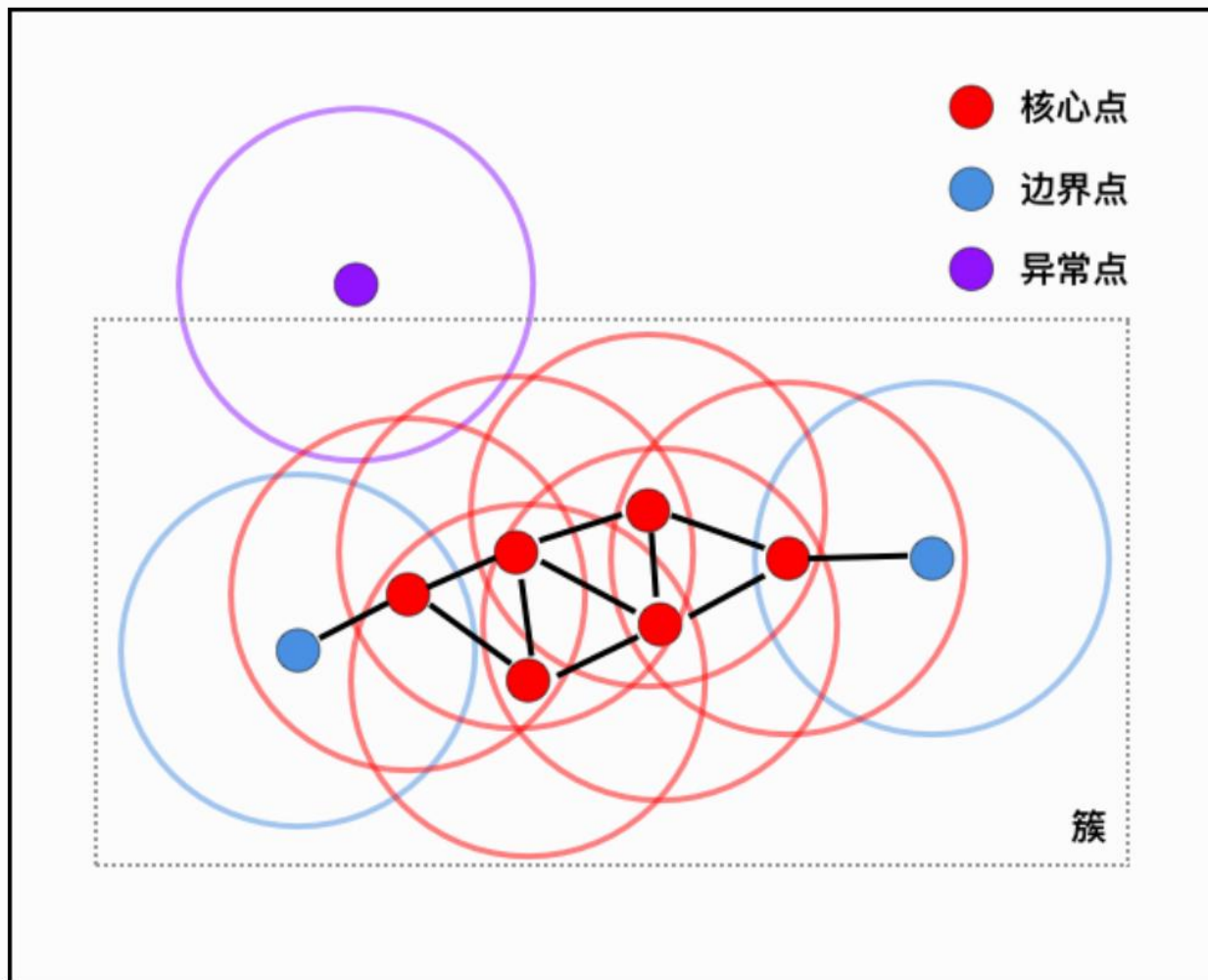
密度聚类--DBSCAN

密度聚类的算法流程

➤ 将距离不超过 $Eps=3$ 的点相互连接，构成一个簇，核心点邻域内的点也会被加入这个簇中。



密度聚类--DBSCAN



密度聚类--DBSCAN

DBSCAN的优点

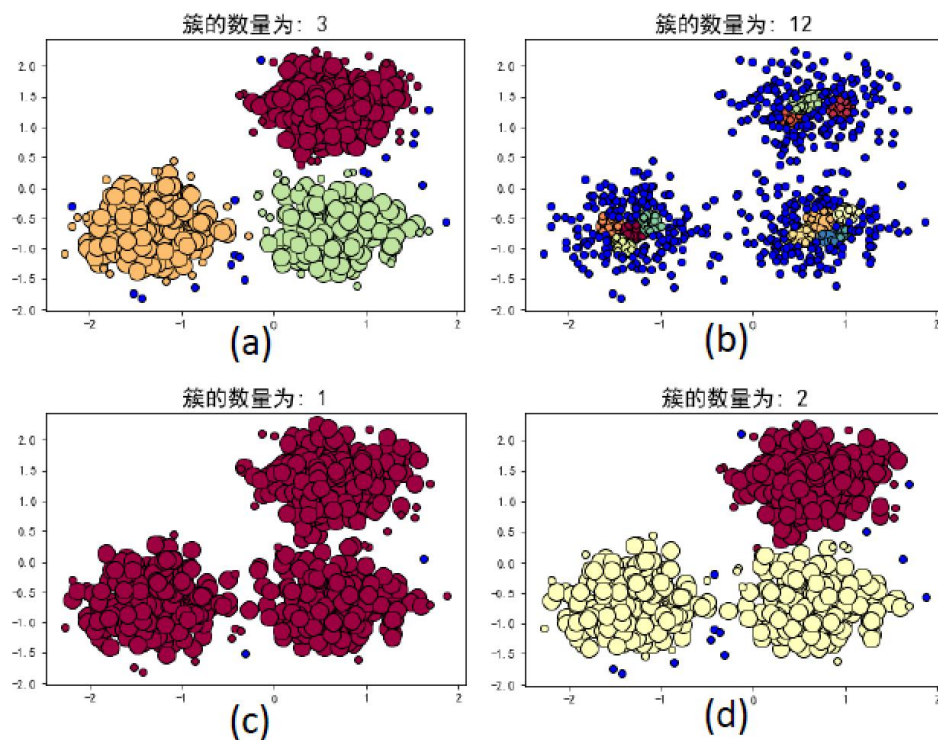
- (1) 可以对任意形状的稠密数据集进行聚类，相对的，K-means算法一般只适合凸数据集；
- (2) 可以在聚类同时发现异常点，对数据集中异常点不敏感。
- (3) 聚类结果没有偏倚，相对的，K-means之类的聚类算法初始值对聚类结果有很大的影响。

DBSCAN的缺点

- (1) 如果样本集的密度不均匀，聚类间距差相差很大时，聚类质量较差，这时用DBSCAN聚类一般不合适；
- (2) 调参相对于传统的K-Means之类的聚类算法稍复杂，主要需要对距离阈值，邻域样本数阈值MinPts联合调参，不同调参组合对最后聚类效果有较大的影响。

密度聚类--DBSCAN

DBSCAN的超参数



DBSCAN超参数案例

图片编号		(a)	(b)	(c)	(d)
评价指标	超参数	eps=0.3 minPts=10	eps=0.1 minPts=10	eps=0.4 minPts=10	eps=0.3 minPts=6
估计的簇的数量		3	12	1	2
估计的噪声点		18	516	2	13
同一性		0.9530	0.3128	0.0010	0.5365
完整性		0.8832	0.2489	0.0586	0.8623
V-measure		0.9170	0.0237	0.0020	0.6510
ARI		0.9517	0.2673	0	0.5414
轮廓系数		0.6255	-0.3659	0.0611	0.3845

这个案例中，当：
eps=0.3，minPts=10的时候，
DBSCAN达到最优效果。

层次聚类

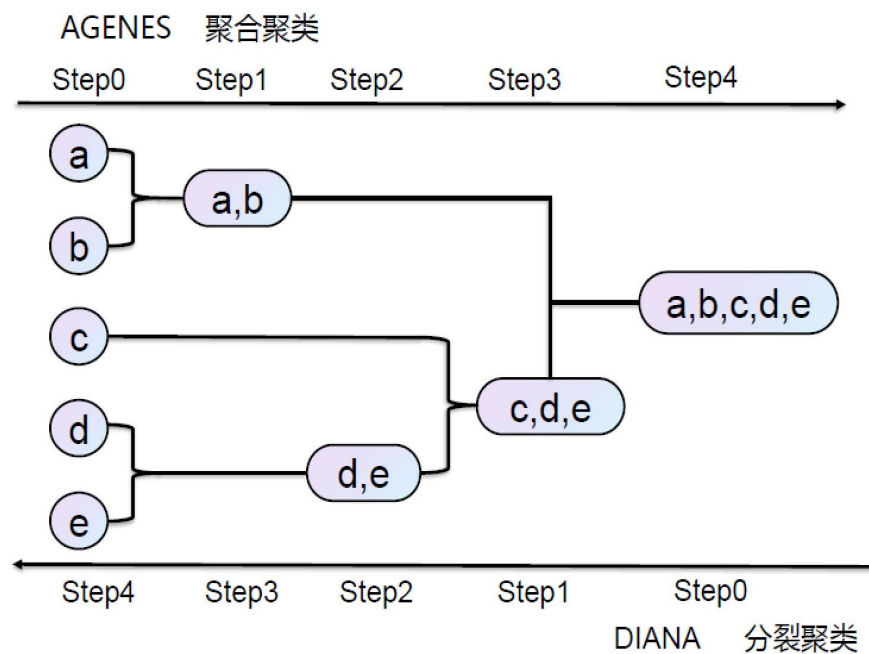
(1) 层次聚类方法将数据对象组成一棵聚类树；

(2) 层次聚类又有聚合聚类（自下而上）、分裂聚类（自上而下）两种方法。

(3) 聚合聚类：一种自底向上的策略，首先，将每个对象作为一个簇，然后合并这些原子簇为越来越大的簇，直到某个终结条件被满足；

(4) 分裂聚类：采用自顶向下的策略，它首先将所有对象置于一个簇中，然后逐渐细分为越来越下的簇，直到达到了某个终结条件；

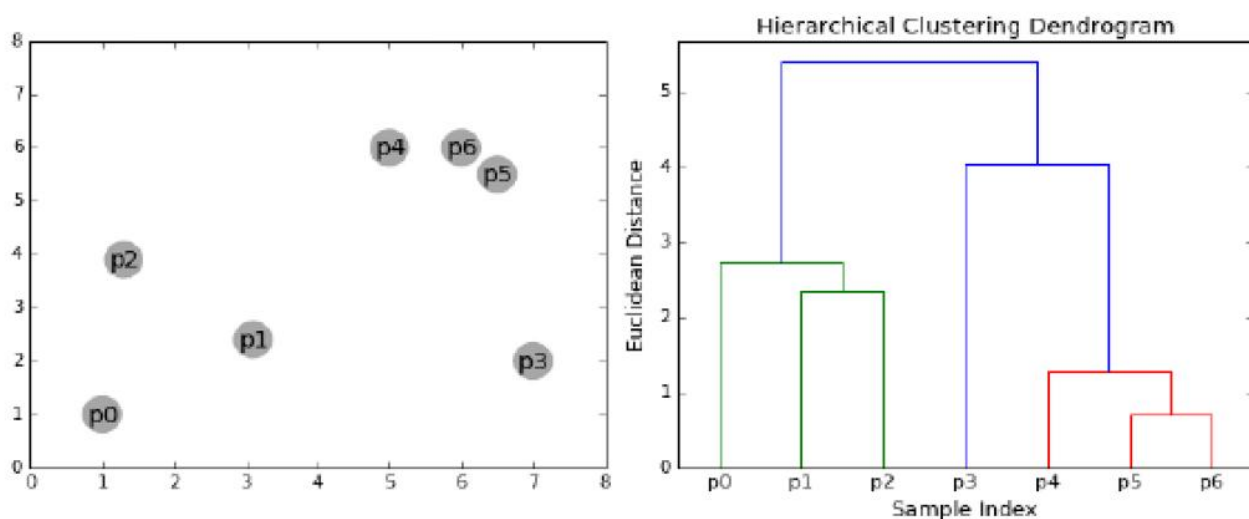
(5) 聚合聚类的代表是AGNES算法；分裂聚类的代表是DIANA算法。



层次聚类结构

层次聚类—Agnes算法

- (1) AGNES算法最初将每个对象作为一个簇，然后这些簇根据某些准则被一步步地合并；
- (2) 两个簇间的相似度由这两个不同簇中距离最近的数据点对的相似度来确定；
- (3) 将距离相近的两个簇合并，建立一个新的簇；
- (4) 聚类的合并过程反复进行直到所有的对象最终满足簇数目。

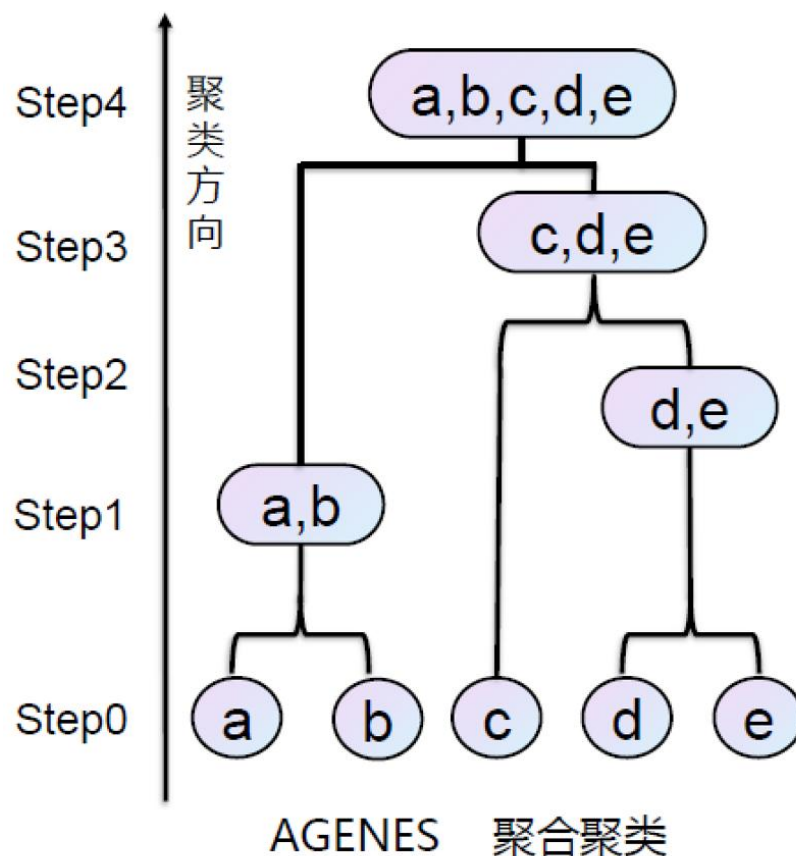


层次聚类—Agnes算法

输入：N个对象，终止条件簇的数目K

输出：K个簇，达到终止条件规定簇数目

- 1) 将每个对象当成一个初始簇；
- 2) Repeat
- 3) 根据两个簇中最近的数据点找到最近的两个簇；
- 4) 合并两个簇，生成新的簇的集合；
- 5) Until达到定义的簇的数目。



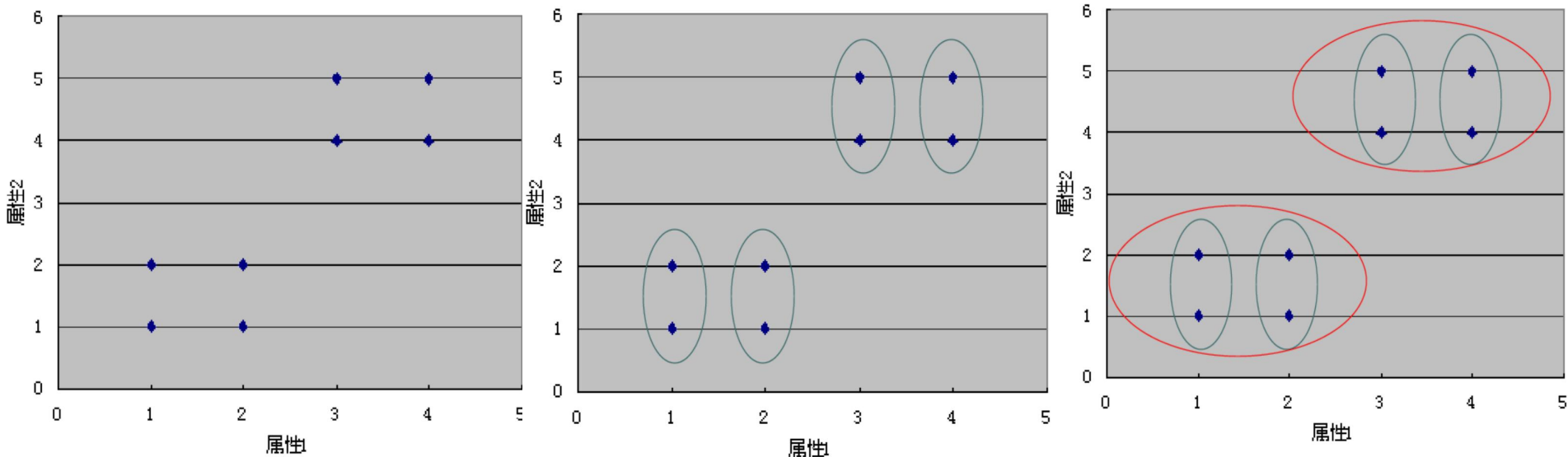
层次聚类—Agnes算法例题

序号	属性1	属性2
1	1	1
2	1	2
3	2	1
4	2	2
5	3	4
6	3	5
7	4	4
8	4	5

- 第1步：根据初始簇计算每个簇之间的距离，随机找出距离最小的两个簇，进行合并，最小距离为1，合并后1,2两个点合并为一个簇。
- 第2步：对上一次合并后的簇计算簇间距离，找出距离最近的两个簇进行合并，合并后3,4点成为一簇。
- 第3步：重复第2步的工作，5,6点成为一簇。
- 第4步：重复第2步的工作，7,8点成为一簇。
- 第5步：合并{1,2}，{3,4}成为一个包含四个点的簇。
- 第6步：合并{5,6}，{7,8}，由于合并后的簇的数目已经达到了用户输入的终止条件，程序终止。

步骤	最近的簇距离	最近的两个簇	合并后的新簇
1	1	{1}, {2}	{1, 2}, {3}, {4}, {5}, {6}, {7}, {8}
2	1	{3}, {4}	{1, 2}, {3, 4}, {5}, {6}, {7}, {8}
3	1	{5}, {6}	{1, 2}, {3, 4}, {5, 6}, {7}, {8}
4	1	{7}, {8}	{1, 2}, {3, 4}, {5, 6}, {7, 8}
5	1	{1, 2}, {3, 4}	{1, 2, 3, 4}, {5, 6}, {7, 8}
6	1	{5, 6}, {7, 8}	{1, 2, 3, 4}, {5, 6, 7, 8} 结束

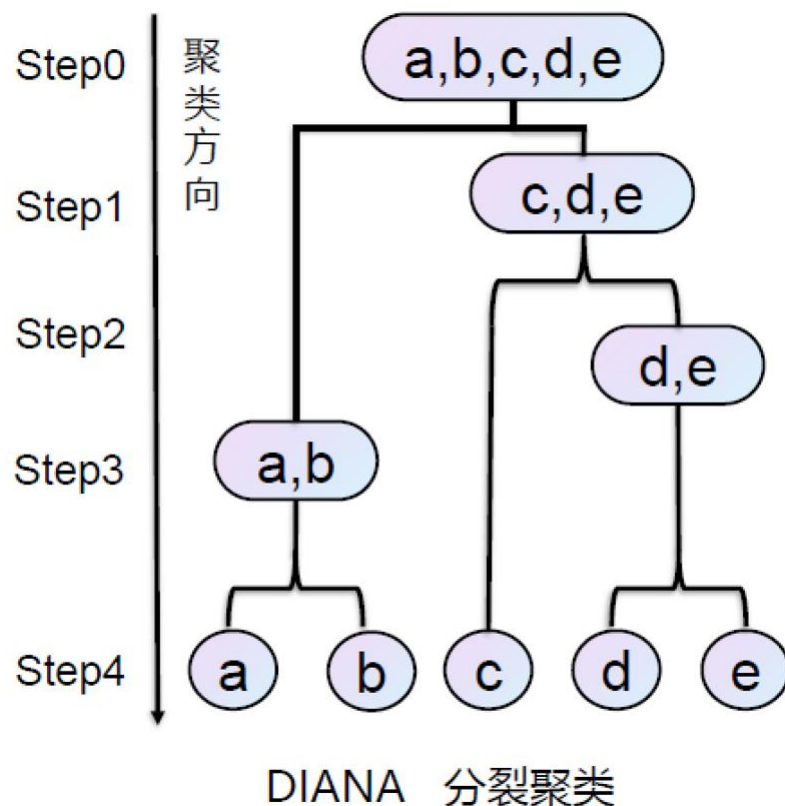
层次聚类—Agnes算法



Agnes算法特点：比较简单，但经常会遇到合并点选择的困难。假如一旦一组对象被合并，下一步的处理将在新生成的簇上进行。已做处理不能撤销，聚类之间也不能交换对象。如果在某一步没有很好的选择合并的决定，可能会导致低质量的聚类结果。

层次聚类—DIANA算法

- DIANA是典型的分裂聚类算法；
- 开始将所有样本分到一个簇；
- 之后将已有类中相距最远的样本分到两个新的簇；
- 重复此操作直到满足停止条件
- 得到层次化的类别



层次聚类—DIANA算法

序号	属性1	属性2
1	1	1
2	1	2
3	2	1
4	2	2
5	3	4
6	3	5
7	4	4
8	4	5

第1步，找到具有最大直径的簇，对簇中的每个点计算平均相异度(假定采用是欧式距离)。
1的平均距离: $(1+1+1.414+3.6+4.24+4.47+5)/7=2.96$
类似地，2的平均距离为2.526;3的平均距离为2.68;4的平均距离为2.18;5的平均距离为2.18;6的平均距离为2.68;7的平均距离为2.526;8的平均距离为2.96。
找出平均相异度最大的点1放到splintergroup中，剩余点在old party中
第2步，在old party里找出到最近的splintergroup中的点的距离不大于到old party中最近的点的距离的点，将该点放入splintergroup中，该点是2。
第3步，重复第2步的工作，splinter group中放入点3。
第4步，重复第2步的工作，splinter group中放入点4。
第5步，没有在old party中的点放入了splinter group中且达到终止条件($k=2$)，程序终止。如果没有到终止条件，因该从分裂好的簇中选一个直径最大的簇继续分裂。

步骤	具有最大直径的簇	splinter group	Old party
1	{1, 2, 3, 4, 5, 6, 7, 8}	{1}	{2, 3, 4, 5, 6, 7, 8}
2	{1, 2, 3, 4, 5, 6, 7, 8}	{1, 2}	{3, 4, 5, 6, 7, 8}
3	{1, 2, 3, 4, 5, 6, 7, 8}	{1, 2, 3}	{4, 5, 6, 7, 8}
4	{1, 2, 3, 4, 5, 6, 7, 8}	{1, 2, 3, 4}	{5, 6, 7, 8}
5	{1, 2, 3, 4, 5, 6, 7, 8}	{1, 2, 3, 4}	{5, 6, 7, 8} 终止

层次聚类—优缺点

➤ 层级聚类的优点

距离和规则的相似度容易定义、限制少

不需要预先制定聚类数

可以发现类的层次关系

可以聚类成其他形状

➤ 层级聚类的缺点

计算复杂度太高

奇异值也能产生很大影响

算法很可能聚成链状

聚类的评价指标

(1)均一性:
$$p = \frac{1}{k} \sum_{i=1}^k \frac{N(c_i == K_i)}{N(K_i)}$$

类似于精确率，一个簇中只包含一个类别的样本，则满足均一性。其实也可以认为就是正确率（每个聚簇中正确分类的样本数占该聚类总样本数的比例和）

(2)完整性:
$$r = \frac{1}{k} \sum_{i=1}^K \frac{N(C_i == K_i)}{N(C_i)}$$

类似于召回率，同类别样本被归类到相同簇中，则满足完整性；
（每个聚簇中正确分类的样本数占该类型的总样本数比例的和）

(3)V-measure:
$$V = \frac{(1 + \beta^2) * pr}{\beta^2 * p + r}$$

均一性和完整性的加权平均

聚类的评价指标

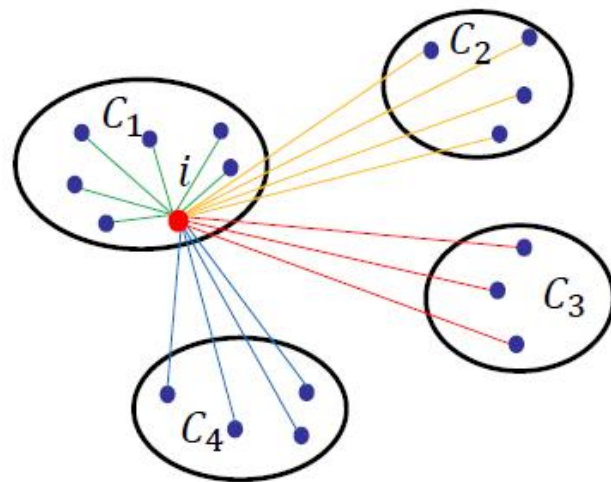
(4) 轮廓系数

样本 i 的轮廓系数：
$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

簇内不相似度： 计算样本 i 到同簇其他样本的平均距离为 $a(i)$ ，应尽可能小。

簇间不相似度： 计算样本 i 到其他簇 C_j 的所有样本的平均距离 b_{ij} ，应尽可能大。

轮廓系数 $s(i)$ 值越接近1表示样本 i 聚类越合理，越接近-1，表示样本 i 应该在边界上；所有样本的 $s(i)$ 的均值被成为聚类结果的轮廓系数。



假设数据集被拆分为4个簇，样本 i 对应的 $a(i)$ 就是所有 C_1 中其他样本点与样本 i 距离平均值；

样本对应的 $b(i)$ 值分两步计算，首先计算该点分别到 C_2 C_3 和 C_4 中样本点的平均距离，然后将三个平均值中的最小值作为 $b(i)$ 度量。

密度聚类--DBSCAN的算法 举例

序号	属性 1	属性 2
1	1	0
2	4	0
3	0	1
4	1	1
5	2	1
6	3	1
7	4	1
8	5	1
9	0	2
10	1	2
11	4	2
12	1	3

下面给出一个样本事务数据库（见左表），对它实施DBSCAN算法。根据所给的数据通过对其进行DBSCAN算法，以下为算法的步骤（设 $n=12$ ，用户输入 $\varepsilon=1$ ，MinPts=4）。

密度聚类--DBSCAN的算法 举例

